

Arquitetura IoT – Rastreamento de Produção Automotiva (com Docker)

1. Objetivo do Sistema

Esta arquitetura tem como objetivo permitir o acompanhamento em tempo real da fabricação e entrega de veículos, desde a linha de montagem até a retirada pelo cliente final.

O sistema utiliza conceitos de Internet of Things, contêineres com Docker e aplicações mobile.

2. Visão Geral da Arquitetura

Fluxo principal do sistema:

Sensores → Gateway IoT → MQTT → Containers (Docker) → Processamento → Banco → API → App

Divisão:

Fábrica

Infraestrutura Containerizada (Docker)

Aplicativo do Cliente

3. Camada 1 – Dispositivos IoT (Chão de Fábrica)

Descrição

Coleta de dados da produção do veículo.

Dispositivos:

RFID no chassi

QR Code

Sensores industriais

CLPs

Câmeras de inspeção

Etapas monitoradas:

Chassi montado

Motor instalado

Pintura

Teste de qualidade

Liberação logística

Dados gerados:

VIN

Status

Timestamp

Envio para o Gateway IoT

4. Camada 2 – Gateway IoT

Função

Ponte entre fábrica e infraestrutura.

Responsabilidades:

Agregar dados

Converter protocolos

Enviar dados para rede

Protocolos:

MQTT

HTTP

OPC-UA

Modbus

Fluxo:

Sensores → Gateway → Broker MQTT

5. Camada 3 – Broker MQTT (Docker)

Descrição

Responsável por receber e distribuir mensagens.

Tecnologias:

Eclipse Mosquitto (container)

EMQX (container)

Executados em Docker

Exemplo de mensagem:

```
{  
  
  "vin": "JTDBR32E720123456",  
  
  "status": "Pintura concluída",  
  
  "timestamp": "2026-03-10 14:20"  
}
```

Fluxo:

Gateway → Broker MQTT (Docker)

6. Camada 4 – Processamento de Dados (Docker)

Descrição

Processamento realizado por microsserviços containerizados.

Etapas:

Validação

Processamento de eventos

Atualização de status

Tecnologias:

Microsserviços (Node.js, Python, Java)

Consumidores MQTT

Containers com Docker

Orquestração com Kubernetes (opcional)

Fluxo:

Broker MQTT → Microsserviços → Banco

7. Camada 5 – Banco de Dados

Função

Armazenamento de dados.

Dados:

VIN

Status atual

Histórico

Previsão de entrega

Tecnologias:

PostgreSQL

MongoDB

MySQL

Rodando em containers Docker

8. Camada 6 – Backend (API)

Descrição

Fornece dados ao aplicativo.

Tecnologias:

Node.js

Java Spring

Python

Executados em Docker

Endpoint:

GET /carro/status/{VIN}

Resposta:

```
{  
  
  "modelo": "Corolla",  
  
  "status": "Em transporte",  
  
  "previsao_entrega": "25/03/2026"  
}
```

9. Camada 7 – Aplicativo do Cliente

Funções:

Acompanhar fabricação

Rastrear transporte

Ver previsão

Status na concessionária

Agendar revisões

Tecnologias:

React Native

Flutter

Swift / Kotlin

10. Funcionalidades do Sistema

✓ Pedido confirmado

✓ Produção iniciada

✓ Montagem finalizada

✓ Teste de qualidade

✓ Em transporte

✓ Concessionária

✓ Pronto para retirada

11. Fluxo do Sistema

Produção inicia

Sensores registram dados

Gateway coleta

Envio via MQTT

Broker (Docker) recebe

Microserviços processam

Banco armazena

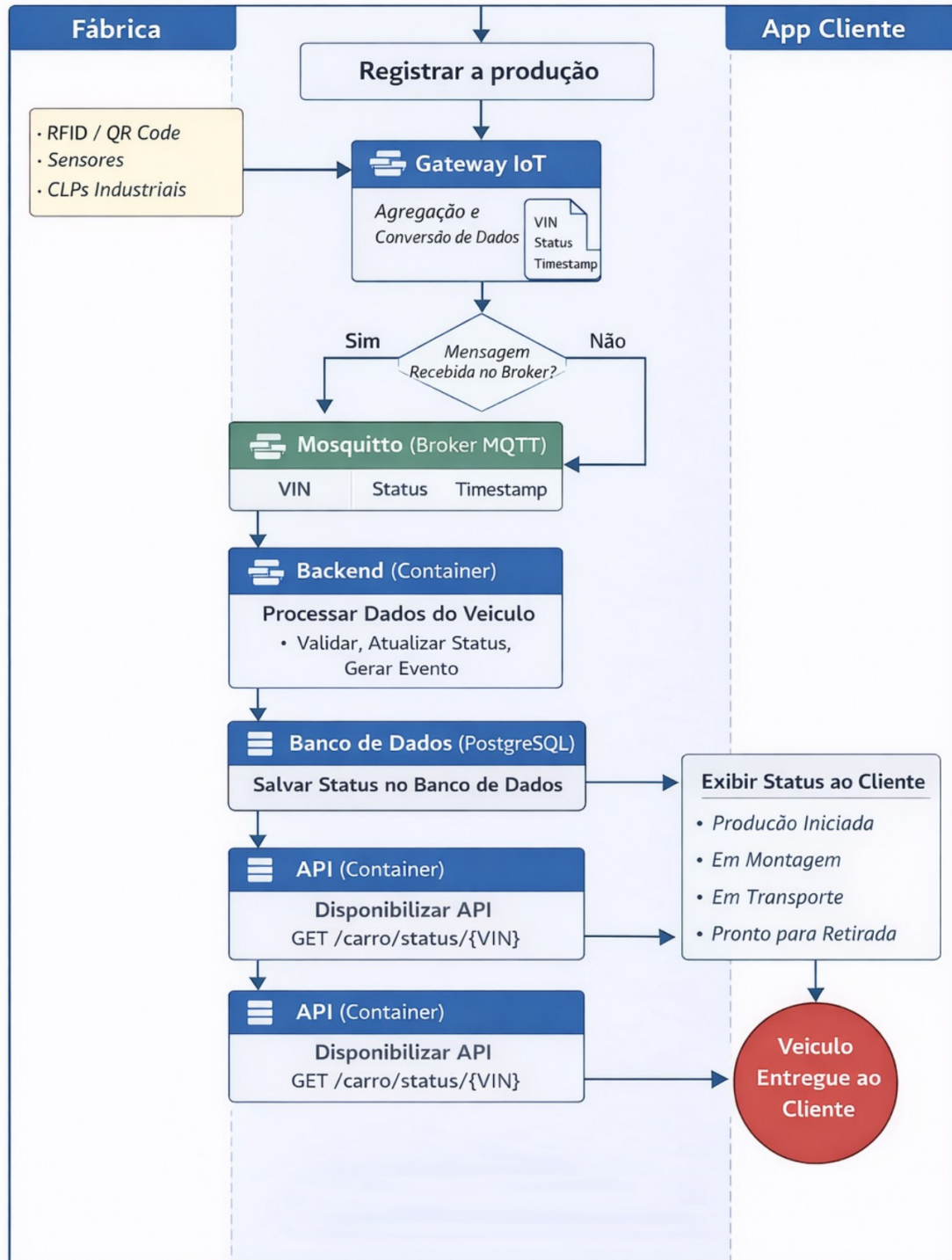
API disponibiliza

App exhibe

Entrega final

12. Arquitetura Final Consolidada

Arquitetura IoT



Sensores → Gateway → MQTT → Docker (Broker + Microserviços + API + Banco) → App

13. Benefícios da Arquitetura

Monitoramento em tempo real

Redução de erros

Alta escalabilidade

Portabilidade com Docker

Fácil implantação

Baixo acoplamento (microserviços)

14. Conclusão

A arquitetura baseada em IoT com uso de Docker representa uma solução moderna e eficiente para o rastreamento da produção automotiva. A containerização permite maior flexibilidade, portabilidade e controle sobre os serviços, eliminando a dependência direta de provedores específicos de nuvem.

Além disso, a integração com Kubernetes possibilita escalar a aplicação conforme a demanda, garantindo alta disponibilidade e desempenho. Essa abordagem facilita a manutenção, reduz custos operacionais e permite rápida evolução do sistema.

Com isso, a solução atende às demandas da Industry 4.0, promovendo automação, eficiência e melhor experiência para o cliente final.

15. Referências

Docker – Documentação Oficial

Kubernetes – Documentação Oficial

OASIS – MQTT Specification

IEEE – IoT Standards

International Organization for Standardization – ISO 23247

Designing Data-Intensive Applications – Martin Kleppmann

Internet of Things

Industry 4.0